Task D6

Name: Kha Anh Nguyen

Student ID: 103814796

Submission Date: 17th August 2025

1. Overview

Inspired by POP MART, FIGR is a Bootstrap + Vue **single-page app** that shows product series with **homepage view** (+ search, type filter)**,** and **checkout** (+ cart and price calculation function). I use **Bootstrap's row-column** with **responsive design styling** to ensure the layout fits well in various devices like phone, iPad and desktop.

For Vue, I dev the app **data** to store **currentView** string variable, so I can apply **view-if** directive for simple view switch between homepage-checkout page view. It also has filter to apply **search filter,** which is like lab P3.2 lookup. In addition, it defines simple **products array** so I can use **v-for** directive to avoid product card repetition at home page. The app data also store a **cart** array for user "add to cart" product, and **form** array to **validate** the user input information at the **checkout** view. For the app functionalities, I dev **computed** functions for checkout process like (**total** price with GST, and **validation** inputs). Moreover, **methods** for actions like add/remove and increase/decrease quantity + inputs are wired with **v-model** so the form stays in sync with state, and I use **v-bind** (e.g., :disabled, :class) to toggle UI states based on those values.

To ensure **accessibility**, I added **labels** for **inputs**, **helper validation** text, <*caption*> for **table**, and **button groups with roles**. I also make sure the HTML code follows indentation conventions

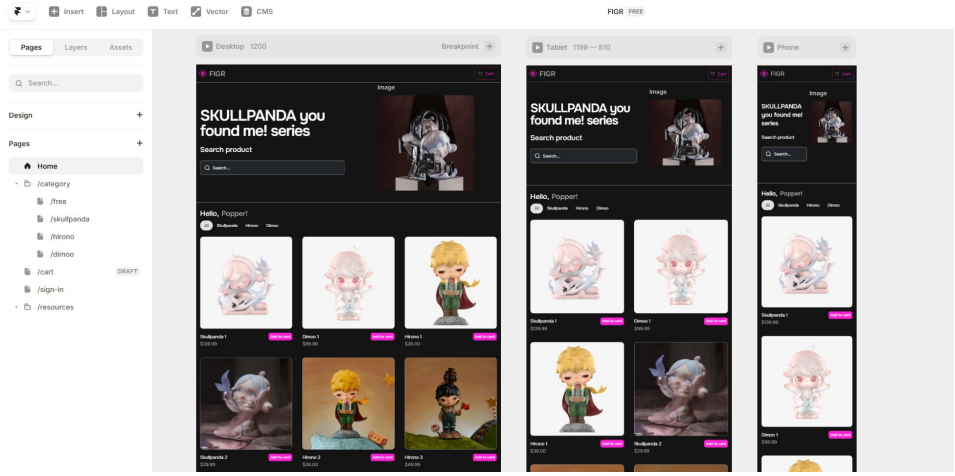
Contents

2. Prototyping

I use Framer to help me with the wireframe and prototyping:

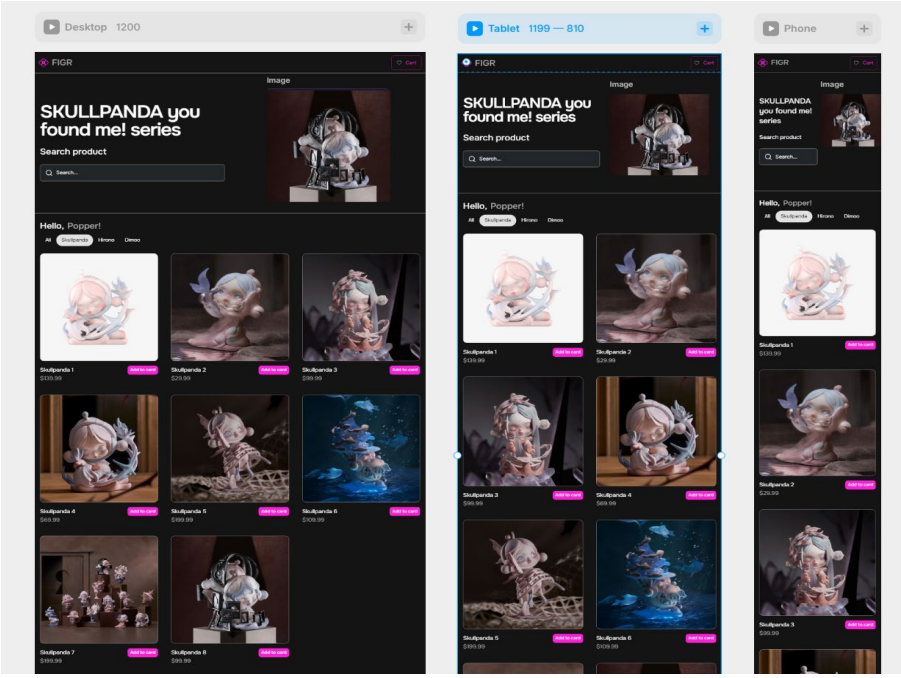
1. Homepage

This is the **homepage** with **search** bar and product types (All or **specific** types) view

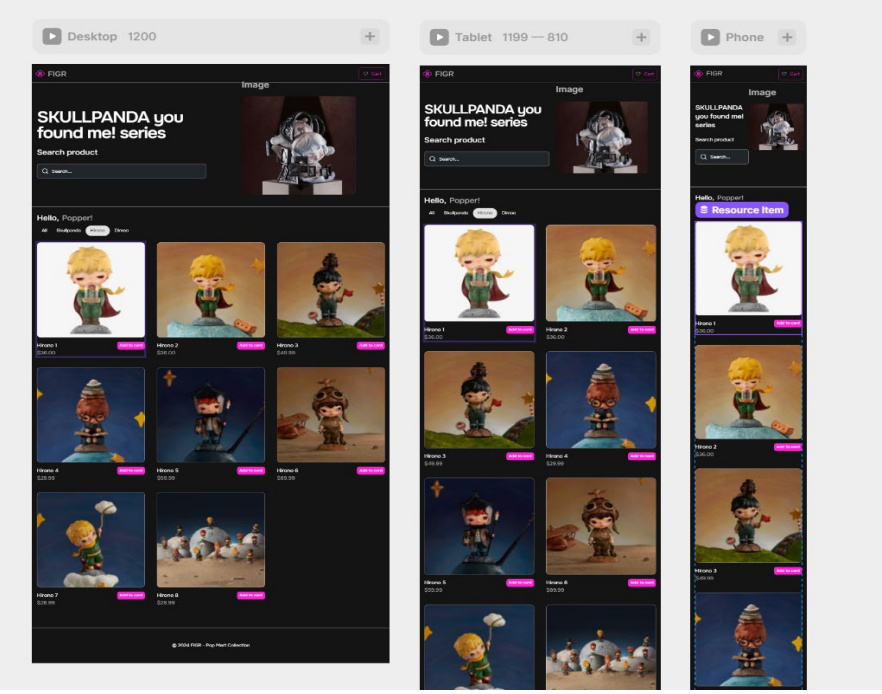


Below are the **specific** type views using simple type filter:

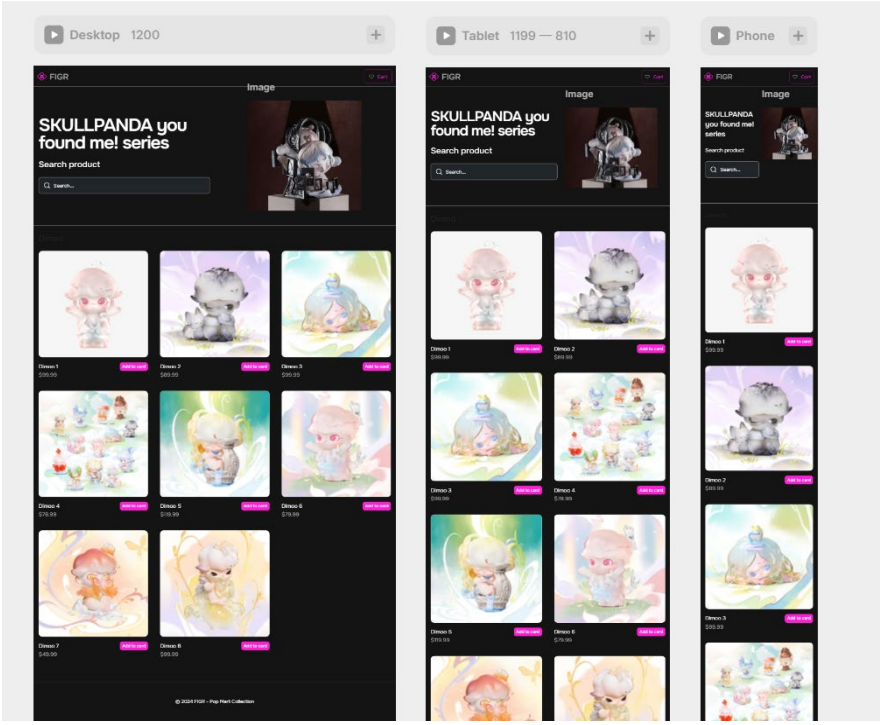
1.1 Filter Skullpanda option



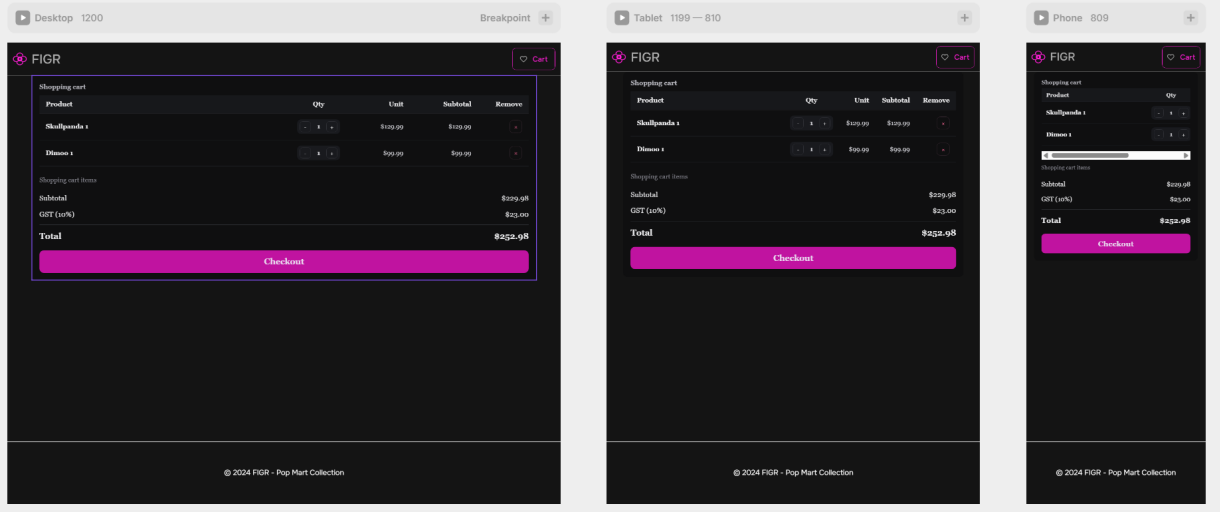
1.2 Filter Hirono option



1.3 Filter Dimoo option



2. Cart view



3. Code implementation

1. Vue instance

Follow the code pattern from previous labs, I created a **Vue app** & **bind** the below **variables to Vue instance** (pseudo code for clearer demonstration).

```
new Vue({
  el: '#app',

  data: {
      // I define app var here, can be string, list, array, etc.
      currentView: 'home',
      filters: { query & type},
      products: [list of dictionary with id, name, price, type & image_path],
      cart: [product index & qty],
      form: { validation props }

  },

  computed: {
      // I add functions for stateful target changes tracking.
  },
  methods: {
      // Methods for stateless functions.
  }
});
```

2. Bootstrap & responsive grid + Vue directives (v-model & v-for)

In conjunction with Bootstrap's responsive design (container, col-6 col-md-4 col-lg-3, etc), I use **rem-based** spacing utilities (mt-2, p-3) so the layout scales across devices **without fixed pixel** widths. Images/cards are fluid (w-100, ratio), and I only use px when needed (like fix outer box):

```
      <!-- Search -->
          <!-- v-model for two-way data binding -->
          <div class="row mt-5">
            <div class="col-12 col-md-10 col-lg-8">
              <label for="home-q" class="form-label">Search product</label>
              <input id="home-q" type="text" class="form-control"
placeholder="Search..." v-model="filters.q" aria-label="Search product">
            </div>
          </div>

      <!-- v-for directive for product repetition -->
      <div class="row g-3">
        <div class="col-6 col-lg-3" v-for="p in filtered" :key="'home-'+p.id">
          <div class="flex-column">
            <div class="mb-2">
              // Product image
              <div class="ratio ratio-1x1">
                <img v-if="p.image" :src="p.image" :alt="p.name" class="w-100 h-
100 object-fit-cover rounded">
              </div>
            </div>
            // Product name & price with add to cart button
            <div class="d-flex justify-content-between align-items-center">
              <div class="fw-semibold">{{ p.name }}</div>
              <button class="btn btn-sm btn-primary mt-2" v-on:click="add(p)">Add
to Cart</button>
            </div>
            <div class="text-muted">{{ formatCurrency(p.price) }}</div>
          </div>
        </div>
      </div>
```

3. View switching (v-if) & navigation

Methods with accept the new view name value and update the app currentView state.

```
  methods: {
    goTo: function(name){
      this.currentView = name;
    },
```

At index.html, simply use **v-if directive** to lazy load the view that matches the app currentView value.

```
    <!-- Home -->
    <div v-if="currentView==='home'">
      <!-- Home content -->
    </div>


    <!-- Checkout Ref: C6.1 registration form validation -->
    <div v-if="currentView==='checkout'">
      <!-- Checkout content -->
    </div>
```

4. Vue Computed

These functions do not accept input parameters, but instead they **cache** and **track** the variables defined in the function to quickly update the variable's state.

```
  computed: {
    // Cart count - automatically updates when cart changes
    cartCount: function(){ return this.cart.reduce(function(a,c){return
a+c.qty},0); },

    // Filtered products - reactive filtering

    filtered: function(){
      var queryLower = this.filters.q.toLowerCase();
      var activeType = this.filters.type;
      return this.products.filter(function(p){
        var isTypeMatch = (activeType==='All') || (p.type===activeType);
        var isQueryMatch = !queryLower ||
p.name.toLowerCase().indexOf(queryLower) >= 0;
```

```
        return isTypeMatch && isQueryMatch;
      });
    },

    // Automatic price calculations update
    total: function(){
      return this.cart.reduce(function(sum,c){ return sum + c.price*c.qty; }, 0);
    }
    gst: function(){ return this.total * 0.10; },
    grandTotal: function(){ return this.total + this.gst; },

    // Ref: C3.4 + C6.1 simple registration form validation with Bootstrap
    // field validity flags
    isFullNameValid: function(){ return /^[A-Za-z\s]+$/.test(this.form.fullName
|| ''); },
    isEmailValid: function(){ return /.+@.+\..+/.test(this.form.email || ''); },
    isMobileValid: function(){ return /^04\d{8}$/.test(this.form.mobile || '');
},
    isPostcodeValid: function(){ return /^\d{4}$/.test(this.form.postcode || '');
},
    canSubmit: function(){
      var f = this.form;
      return !!(this.isFullNameValid && this.isEmailValid && this.isMobileValid
&& f.street && f.suburb && this.isPostcodeValid && this.cart.length>0);
    }
```

5. Vue Methods

These functions accept input parameters, but **do not store the parameters' value**, they simply execute their job with any parameter values when called (**system** does **not** need to **store** and **track** these redundant values, reduce unnecessary state writes and recomputes).

```
  methods: {
    goTo: function(name){
      this.currentView = name;
    },
    formatCurrency: function(n){ return '$' + n.toFixed(2); },

    // Cart management
    add: function(p){
      var idx = this.cart.findIndex(function(c){ return c.id===p.id; });
```

```
      if(idx===-1){ this.cart.push({ id:p.id, name:p.name, price:p.price, qty:1
}); }
      else { this.cart[idx].qty += 1; }
    },

    // Quantity management
    incQty: function(i){ this.cart[i].qty += 1; },
    decQty: function(i){ if(this.cart[i].qty>1) this.cart[i].qty -= 1; },
    remove: function(i){ this.cart.splice(i,1); },
    submitOrder: function(){ alert('Submitted! (mock)'); this.currentView='home';
  }
```

6. Accessibility Implementation

I add **<label>** for **inputs** and **helper** validation text. Also, the cart **table** has a **<caption>** and **headers**, action buttons include **aria-label** (increase, decrease, remove). **Dark theme** colors meet **contrast pink highlight.**

```
          <input id="home-q" type="text" class="form-control" placeholder="Search
..." v-model="filters.q" aria-label="Search product">
        </div>

          <table class="table table-dark align-middle">
            <caption class="text-secondary">Shopping cart items</caption>
            <thead>
              <tr>
                <th scope="col">Product</th>
                <th scope="col">Qty</th>
                <th scope="col" class="text-end">Unit</th>
                <th scope="col" class="text-end">Subtotal</th>
                <th scope="col">Remove</th>
              </tr>
            </thead>
            <tbody>
                <!—ARIA labels -->
               <button class="btn btn-outline-primary" aria-label="Increase
quantity" v-on:click="incQty(i)">+</button>
                <button class="btn btn-sm btn-outline-primary" aria-
label="Remove item" v-on:click="remove(i)">Remove</button></td>
              </tr>
            </tbody>
          </table>
```

7. HTML5 coding conventions

I follow **HTML5** conventions: <!DOCTYPE html>, lang="en", **UTF-8**, and the responsive meta viewport. **Indentation** is **consistent** so the structure reads cleanly. I load **Bootstrap** and **icons** via **CDN**, keep **scripts** at the **end**, and separate concerns (HTML for structure, JS for logic). This keeps the document valid and easy to maintain.

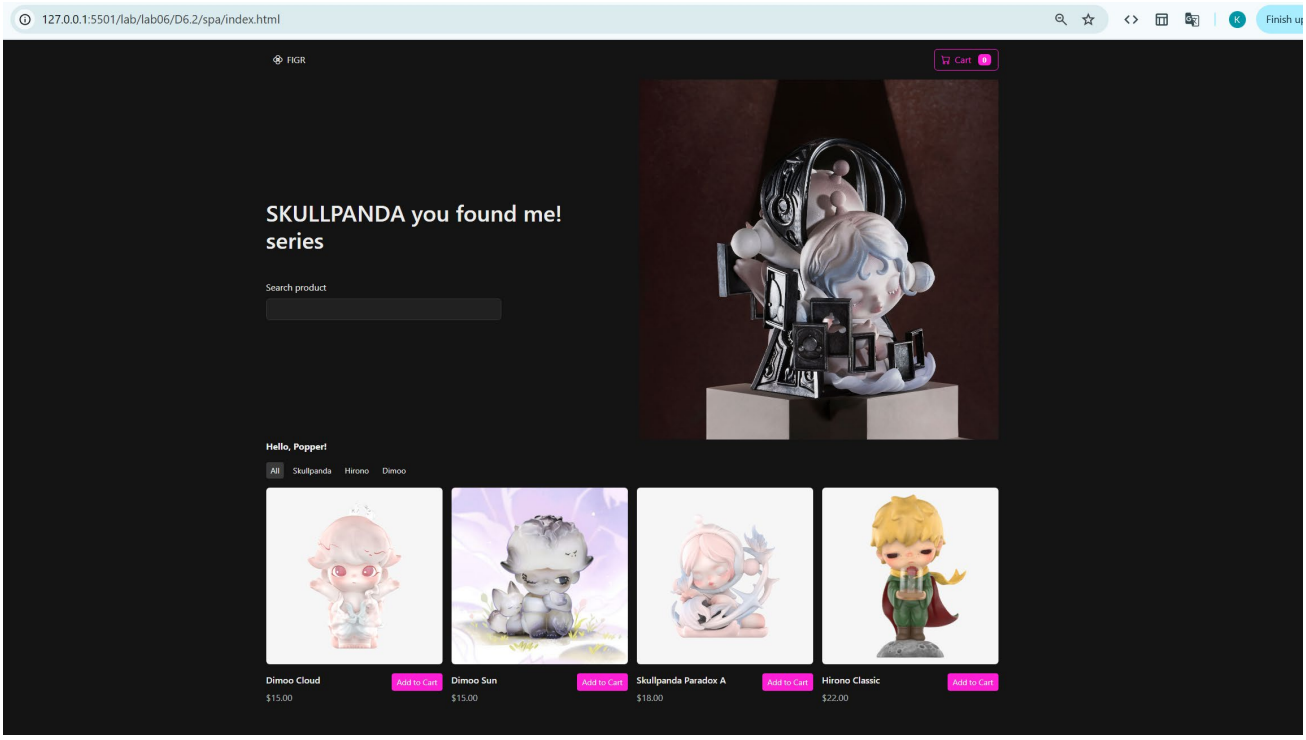
```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>FIGR</title>
  <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/css/bootstrap.min.css"
rel="stylesheet">
  <!-- Bootstrap Icons -->
  <link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/bootstrap-
icons@1.11.3/font/bootstrap-icons.min.css">
</head>

<body data-bs-theme="dark" class="p-3">
  <div id="app" class="container">
    <!-- Content-->
  </div>

  <!-- Footer -->
  <footer class="mt-5 py-3 text-center text-muted">
    <small>&copy; 2024 FIGR - Pop Mart Collection</small>
  </footer>

  <script src="https://unpkg.com/vue@2.6.14/dist/vue.js"></script>
  <script src="app.js"></script>
</body>
</html>
```

4. Web Results

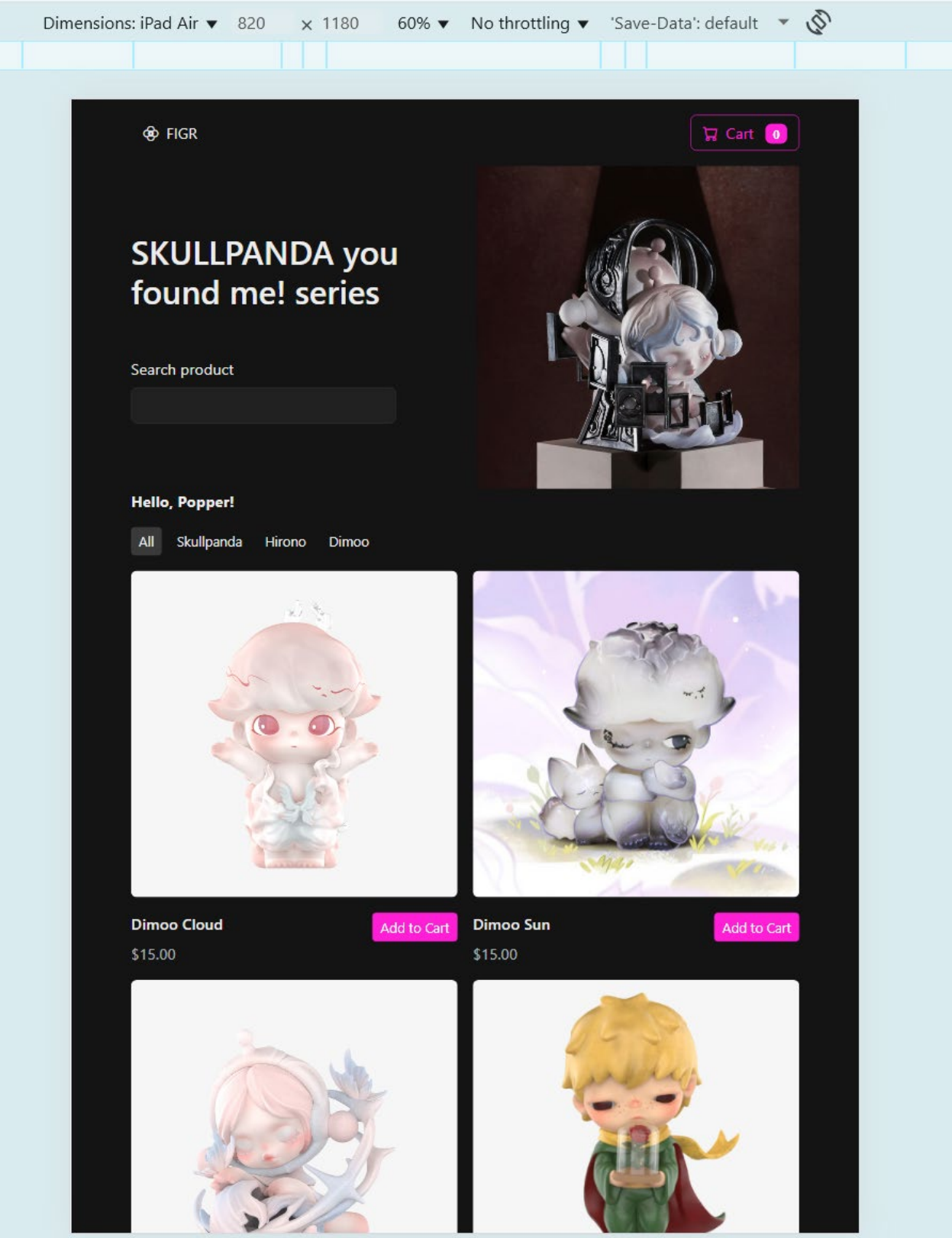
Below is the final web result:

1. Homepage

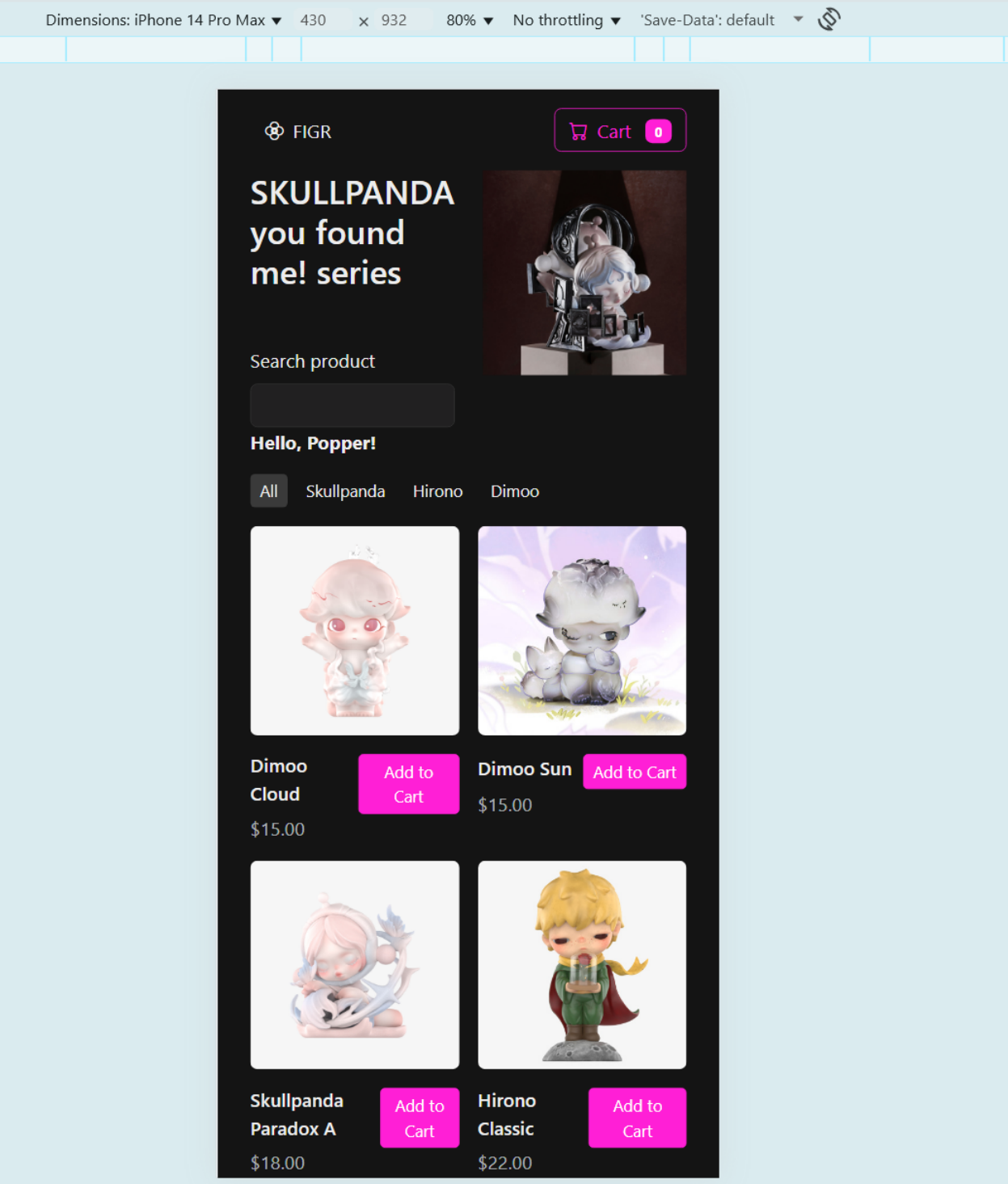
1.1 Large desktop view



1.2 Medium ipad view

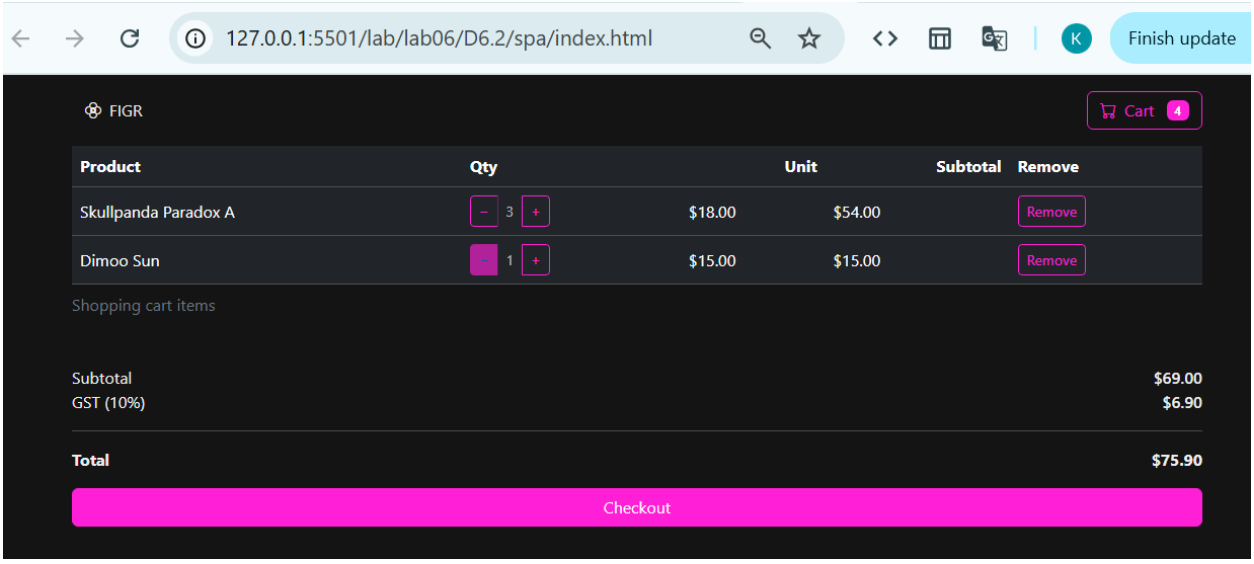


1.3 Small phone view

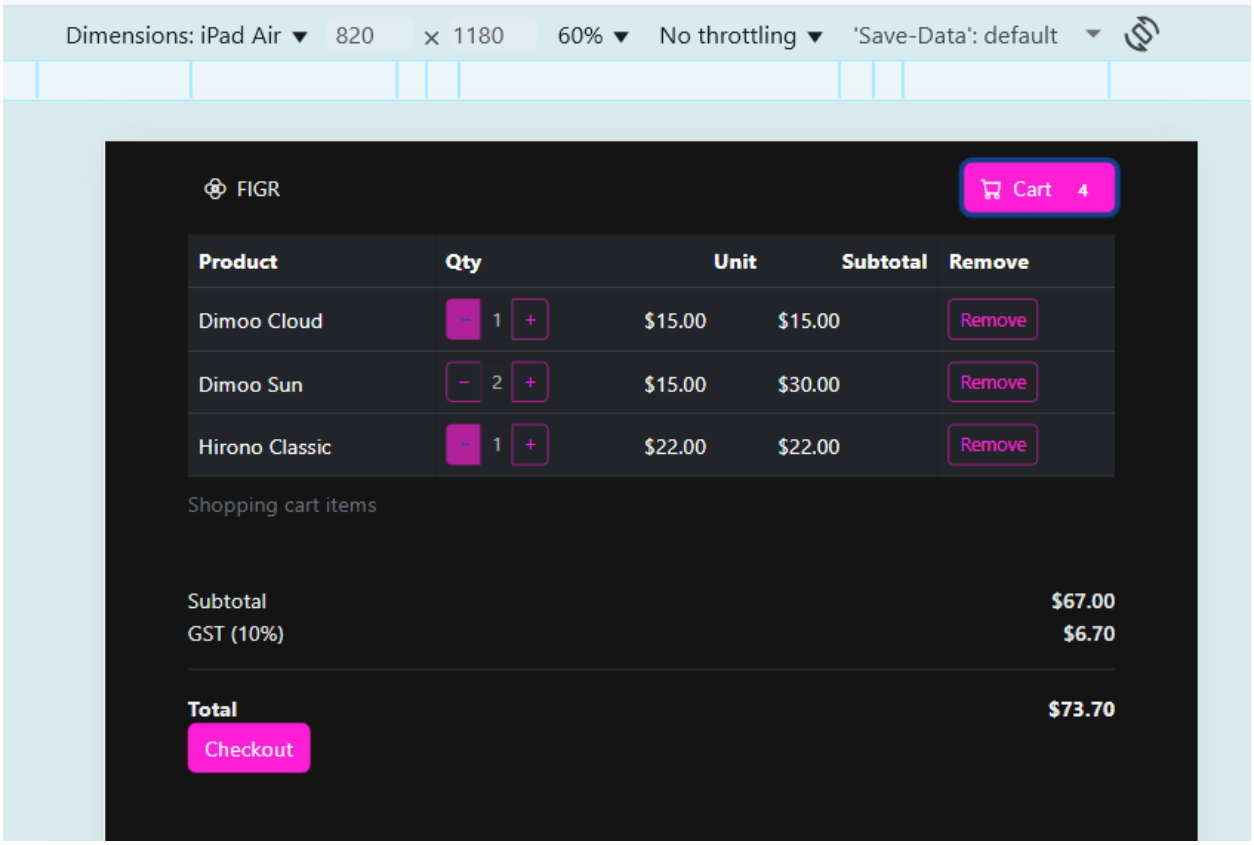


1. Cart view

2.1 Large desktop view



2.2 Medium ipad view



2.3 Small phone view